

B. y33tflag

Kita diberikan sebuah script python yang mengambil flag rahasia, partisikan menjadi chunk 12-byte, buat chunk menjadi bilangan bulat, dan hitung akar bilangan bulat tersebut. Tantangannya, kita hanya diberikan bagian pecahan saja (bilangan setelah titik atau koma) dari akar dengan presisi sangat tinggi (2000 digit).

Tujuan: Diberikan ekspansi desimal .15825..., carilah bilangan bulat S sehingga $\sqrt{S} = \text{sesuatu}.15825\dots$

Intuisi: Mengapa bisa diselesaikan?

Secara sekilas, tantangan ini kelihatan mustahil. Ada tak hingga bilangan yang memiliki desimal tersebut. Namun, terdapat dua konstrain:

1. Bilangan awal S adalah **bilangan bulat**.
2. Bagian integer dari akar (k) relatif kecil (muat di dalam 12 byte).

Marilah kita melakukan aljabar sederhana.

Misalkan S adalah bilangan bulat rahasia.

Misalkan k adalah bagian integer dari $\sqrt{S}$ (yang kita tidak ketahui).

Misalkan α adalah bagian pecahan (yang kita ketahui).

Kita bisa tulis akar sebagai berikut:

$$\sqrt{S} = k + \alpha$$

Jika kita kuadratkan kedua sisi, kita dapat berikut:

$$S = (k + \alpha)^2$$

$$S = k^2 + 2k\alpha + \alpha^2$$

Kita bisa susun ulang persamaan ini. Berhubungan dengan S dan k adalah bilangan bulat, $S - k^2$ juga merupakan bilangan bulat. Kita sebutkan ini sebagai M .

$$M = 2k\alpha + \alpha^2$$

Kita perlu mencari bilangan bulat k (koefisien) dan sebuah bilangan bulat M (hasil) yang memenuhi relasi untuk α .

Metode Grid

Untuk menyelesaikan masalah ini, kita butuh teknik **Lattice Reduction** (lebih tepatnya algoritma LLL). Bayangkan kita berada di suatu titik origin $(0,0,0)$ dari graf 3D. Kita diberikan tiga panah (vektor) berbeda dengan panjang dan arah berbeda. Kita diperbolehkan untuk memperpanjang atau putar arah dari vektor dengan bilangan bulat dan gabungkan mereka.

Tujuan kita adalah menggabungkan semua panah supaya letaknya sedekat mungkin dengan titik origin tanpa berada di titik origin. Kita bisa menulis ulang persamaan kita supaya menyerupai target pada grid:

$$k(2\alpha) + 1(\alpha^2) - M(1) \approx 0$$

Ini menyerupai kombinasi linier! Kita ingin mencari bilangan bulat k , 1 , dan M sehingga saat kita kalikan mereka dengan nilai tertentu, hasilnya saling kanselasi menjadi nol.

Setup

Kita susun matriks basis. Berhubung dengan komputer bisa menangani bilangan bulat lebih baik daripada desimal tak hingga, kita skalakan dengan bilangan besar (kita sebut $C = 10^{1000}$).

Kita buat tiga vektor untuk variabel kita:

1. Vektor 1 (untuk k): mewakili variabel 2α .
2. Vektor 2 (untuk 1): mewakili variabel α^2 .
3. Vektor 3 (untuk M): mewakili variabel -1 .

Saat kita jalankan **Algoritma LLL** pada vektor ini, ia bekerja sebagai kompaktor sampah ajaib. Ia rotasikan dan reduksi vektor untuk mencari kombinasi terpendek yang mungkin dari mereka. Berhubungan dengan rahasia S menghasilkan relasi matematika sempurna, vektor

terpendek yang ditemukan oleh LLL akan korespondensi tepat dengan persamaan kita. Koordinat untuk vektor pendek akan mengungkapkan k kita yang tersembunyi.

Solusi

Saat kita memiliki k, bagian sisanya cukup mudah. Kita tinggal substitusikan ke rumus berikut:

$$S = \text{round}((k + \alpha)^2)$$

Kemudian kita ubah S menjadi byte untuk mendapatkan chunk flag.

Catatan: Karena angka masif 10^{1000} , float standar komputer akan gagal. Kita perlu menggunakan pustaka arbitrary-precision seperti mpmath.

Kode B.1: Script Solver Challenge

```
import mpmath as mp

# 1. Set precision extremely high.
# We are working with lattice constants ~10^1000.
# Standard floats overflow at 10^308. We need mpmath for
everything.
mp.mp.prec = 10000

output_data = [

'15825840091281348126520367062897437019036818888101321493485552161
525948259012155738965958740639448471275429546230372438615927635122
985330958176584638197902072895873338661085155157210759176638865823
518310438085035061229977391514127199712226795526221069182913239232
715044297964663111490654153135613774013064425173807214562643702444
722917776013860094874186871555923136484174667750185067087134236982
321444075418889987051919003241660121864538331792377276896373073063
147366759549514746487043579689248956073313624195634930477249219589
36956311053484972147115497179787411162784292946092332682967',

'19845218993523847823759331519960663424932065864518374645033557548
437150012812773300571531076208261546024767128737345752347807871180
```

870512382732087654815453876170354896767592284678071981746502770275
736219618137672358894699463627147589987944868598330687654298560633
195626215632975222520574818421512958361136772219663932121214904548
392479801226004145975264583063545418508231446489635115611634714446
771447390478922392403409584886959248893229013724476581576280945380
096602412274116864818748729616057685918452472011055562699283131115
96843052813339173335524263777938480316289472067091910205786' ,

'17545925302154403446276930146984730447881149497607209403786107962
350988731703864238260641720398159395598979159168102742000477115943
714887016580830875643451394594009093645802020929366116887039354670
423561566670386465956826425178521267460599289588122227935405254027
332102777547668200774164397489451492158294419699426396677337024754
241192558557502405475103804057650358675199700980229373114151312630
124412692753051192843980657958484564731735634681415841022233758527
833633964841154833759123554871308013850016298695481520719927133709
47694195871775475506635661191833768801006369445490445838879' ,

'94807834839534545013530238049497348522819086922738751946193945327
220424281833181975072018076158194665279514544372854718586224877096
796233002785612245569937691487135837195271097503713327707570305998
790019285746295901992353624970430601915408527207137900910568294951
166182885057256650050586283911779300646948642980391260206224815303
324223066763259689726299420270552535349347684080935464111507758943
007981383675245105050257854274401354667382319634405079002387559293
333931123022744900756670632267167142287224332067012840934795873505
71057261880399154720161385815398745785445557358134796363534' ,

'99377138303063685671136711150456426296553189761632223092957837280
656572873970800552397177300651909981747627885894287460202089289397
985385228444191550603406858095494414084213365538033444869714414732
980035042080561906427641294904360616228201666276928827921160594480
006485702076459562174318750264898233168777823686726244726317596837
733867766487819186620968690919719490305591578906889628906890776834
511815653767144432456036606006997583056467275509454032463043982919
959296815541800353626338532097521346898629156760645620496680555257
62107220043261847609840337225153213431311731822822953503029' ,

'28869818091177431137351648651386036713489301473603866560325541869
388072001449566624506361426372718445719203293873281464779007473347
714496934562248094014501833077074258623203971483210415191332211244
521993547397302992033527572194134722398667928174191303828295779909

297744350013783253853314580556538484619932985384320457815636031956
624573317455328107610794882054380666693455585470315584086759865759
089151780086366703946930463098736264584216568703396558844401932574
540862686114696569725260218263608266886218139833282880496392264413
95497008550566198839267360264522637673141341277420199648575' ,

' 37862235034807966004139633971955239877275244983016523700989852071
493641536607670498794380672634364607516266694760818762273801508119
689012729154464580649394903607527097240610410001980203008490577270
925899571662612289650735693885532765140689856512258922670306667521
668912903409611374431870066042191033304657879384512536241668868921
578708267646641987484300420283042927756691743372975189332371729943
715585627599701544899621441515844053017465397114523302078679178687
467377891712521610005562960003652488946185939530681675802546069701
34244815140024461353276768631183924580571111585618798528394' ,

' 77275973881965028401002205198716049366657029383433864784157637625
755332096665550531393743554854052856945987002540729887215563408634
265688505920844365856831926294866584143049894206951938726564106189
070812900903836896046147073940558955982685318397169843903363760202
250021831392261590939639428291434184742726686772793224818389969865
740440994237836634344994004965812949740793958311976624795638375515
788795116444019741803526385885949964103005071860510321902360899417
886661551513966337857287325237431343945181623411587568141651161751
82469259583858474844273319864954244087084854632617533724193' ,

' 02198576678084543931313170446471227688766118675200018978236508020
359801234265451332310888816591328452273310813386243971388091782844
776785326599942715656290717903098194313006511914294670518380621847
358499964139566954111402360375691559276226586721167594966242622548
22712566212368191125764413801637333619120598595756296745134805693
809642329646722664166467412378825969397342532884029697332601562578
428725229939878633000219334669888617378893611747508997676091574714
095237680845399368167356486943055675253214365285440458908193545838
12716223323576594470515128244564373677498381293618158062431' ,

' 39570930505359988972374842710194944891308588088559282731797006068
539110967798185683429011213463118755163063254658434254019739147829
298847820114190048414329977686957211583786757129985858934045292622
742271814338600273388408929088555306802049848876019081570588806809
403685601859304812307952489504513398042030937884363477064373279827
076335571869607612361162290225437966448529492803433790296341118759

```
225364767052912188241680114169737548982198470627333679227196916329
453605343132569754439473303981977580701657028620570374554315267295
05595099594217213493508486258047720167371274812072513274771'
]
```

```
def lll_reduction(basis):
    """
    Robust 3D LLL reduction using mpmath to prevent overflow.
    Returns the reduced basis.
    """
    n = len(basis)
    # Convert basis to mpf for internal GS coefficients, keep
    basis as integers in main flow
    basis_f = [list(map(mp.mpf, row)) for row in basis]

    # Gram-Schmidt Orthogonalization (GS)
    b_star = [row[:] for row in basis_f]
    mu = [[mp.mpf(0)] * n for _ in range(n)]

    def update_gs():
        for i in range(n):
            for l in range(3):
                b_star[i][l] = basis_f[i][l]
            for j in range(i):
                # Using dot product of mpf values
                dot_val = sum(basis_f[i][l] * b_star[j][l] for l
in range(3))
                norm_j = sum(b_star[j][l]**2 for l in range(3))
                if norm_j == 0:
                    mu[i][j] = mp.mpf(0)
                else:
                    mu[i][j] = dot_val / norm_j
            for l in range(3):
                b_star[i][l] -= mu[i][j] * b_star[j][l]

    update_gs()
    k = 1
    delta = mp.mpf('0.99')

    while k < n:
        # Size reduction
        for j in range(k - 1, -1, -1):
```

```

        if abs(mu[k][j]) > 0.5:
            # Use mp.nint then cast to int to stay safe and
precise
            q = int(mp.nint(mu[k][j]))
            for l in range(3):
                basis[k][l] -= q * basis[j][l]
                basis_f[k][l] -= q * basis_f[j][l]
            update_gs()

        # Lovasz condition
        norm_k = sum(b_star[k][l]**2 for l in range(3))
        norm_k_1 = sum(b_star[k-1][l]**2 for l in range(3))

        if norm_k >= (delta - mu[k][k-1]**2) * norm_k_1:
            k += 1
        else:
            basis[k], basis[k-1] = basis[k-1], basis[k]
            basis_f[k], basis_f[k-1] = basis_f[k-1], basis_f[k]
            update_gs()
            k = max(k - 1, 1)

    return basis

flag_bytes = b""

for i, s_str in enumerate(output_data):
    # s_str is alpha
    alpha = mp.mpf("0." + s_str)

    # The relation is  $s = (k + \alpha)^2 = k^2 + 2k\alpha + \alpha^2$ 
    #  $s - k^2 = 2k\alpha + \alpha^2$ 
    # Let  $M = s - k^2$  (an integer).
    # Then  $2k\alpha + \alpha^2 = M$ 
    #  $\Rightarrow 2k\alpha + 1\alpha^2 - M \cdot 1 = 0$ 

    # We build a lattice to find integers (k, 1, M).
    # Lattice Basis:
    # v1 = [1, 0, C * 2*alpha]    (Coefficient for k)
    # v2 = [0, 1, C * alpha^2]    (Coefficient for 1)
    # v3 = [0, 0, -C]             (Coefficient for M)

    C = 10**600 # Scaling constant matching roughly the precision

```

```

basis = [
    [1, 0, int(2 * alpha * C)],
    [0, 1, int(alpha**2 * C)],
    [0, 0, -C]
]

reduced = lll_reduction(basis)

chunk_found = False

# Check all rows in reduced basis for the vector of form (k,
1, epsilon)
for row in reduced:
    # We expect the 2nd component (index 1) to be +/- 1
    if abs(row[1]) == 1:
        k_cand = abs(row[0])

        # Reconstruct s: s = round((k + alpha)^2)
        val = (mp.mpf(k_cand) + alpha) ** 2
        s = int(mp.nint(val))

        # Verify correctness
        # We use enough precision to differentiate
        check_str = str(mp.sqrt(s)).split('.')[1]
        if s_str.startswith(check_str[:50]):
            try:
                # Convert to bytes (defaulting to big endian
for crypto)

                chunk = s.to_bytes(12, 'big')
                flag_bytes += chunk
                print(f"Recovered chunk {i+1}: {chunk}")
                chunk_found = True
                break
            except:
                pass

        if not chunk_found:
            print(f"Failed to recover chunk {i+1}")

print("\nFinal Flag:",
flag_bytes.decode(errors='ignore').rstrip('X'))

```

Kesimpulan

Dengan menyadari bahwa $2k\alpha + \alpha^2$ harus berupa bilangan bulat, kita mengubah masalah tebakan menjadi **Shortest Vector Problem (SVP)**. Algoritma LLL bisa menyelesaikan masalah ini dalam dimensi rendah secara efisien, mengizinkan kita untuk mendapatkan flag hampir secara instan.

Flag:

`XMAS{this_is_actually_an_exercise_problem_from_the_book_An-Introduction-To-Mathematical-Cryptography_by_silverman}`